HarvestOS

Every harvest deserves a market.

MASTER PRODUCT & DEVELOPMENT ROADMAP

Product strategy • UX architecture • Complete functional modules

PostgreSQL schema • Intelligence logic • APIs • Security

48-hour hackathon MVP • Production roadmap • Deployment

HACKJOS 2026 / PLATEAU STATE, NIGERIA

Version 1.0 • October 2026 • Build-ready planning document

THE MASTER BLUEPRINT

HarvestOS is a coordination and decision-intelligence operating system for perishable agricultural produce. The platform helps farmers, buyers, storage operators, transporters and coordinators act before market-access failures turn harvests into financial and physical losses.

Core product principle: Build a dependable system of record first, a transparent intelligence layer second, and executable coordination workflows third. A recommendation is not a successful transaction; an accepted offer is not rescued food; only verified outcomes can support impact claims.

Document contents

PART	CONTENTS
01 | Strategy	Vision, problem statement, users, objectives and milestones
02 | Experience	Identity, palette, typography, page architecture and stakeholder journeys
03 | Architecture	Technology stack, infrastructure, folder structure and security model
04 | Product modules	Farmer, buyer, transport, storage, coordinator, admin and public site
05 | Intelligence	Urgency scoring, matching, allocation and impact measurement
06 | Data + API	PostgreSQL model, migrations, endpoints, concurrency and media
07 | Delivery	Testing, hackathon MVP, ten-week rollout, deployment and commercialization
08 | Appendices	Code examples, checklists, acceptance criteria and research references

Document status: Detailed implementation roadmap, not a completed working software product. Scores, weights, example harvests, sample records and pilot metrics are proposed assumptions until verified in the field.

01 | Business and product strategy

1.1 Vision, mission and positioning

Vision: Agricultural supply chains in which valuable harvests are discovered, allocated and transported before avoidable delays destroy value. Mission: Give smallholder farmers and agricultural businesses a reliable shared operational layer for crop supply, buying, storage and distribution.

Positioning: HarvestOS differs from a simple product listing marketplace by identifying at-risk quantities, evaluating feasible alternatives and coordinating real-world follow-through. Initial geography: Plateau State. Initial produce: tomatoes. Subsequent crops: Irish potatoes, cabbage, pepper and others after process validation.

1.2 Problems to solve

PROBLEM	PRODUCT RESPONSE
Short selling windows	Pre-harvest supply registration, early demand, urgency queue
Weak buyer access	Verified demand requests, compatible offers and buyer discovery
Transport fragmentation	Quotes, grouping opportunities, collection and delivery records
Storage scarcity	Partner facility directory, reservations and capacity checks
Uncertain margins	Net proceeds comparison including applicable costs
Poor records	Orders, inventory movements, payments and dispute history
Unverified impact	Quantity and transaction evidence plus comparison methodology

1.3 Outcomes and operating metrics

- Harvest quantity sold, physically lost, rejected and still held; track by crop and actual mass where known.
- Net farmer revenue per verified kilogram, accounting for transport, storage and fees.
- Supply-to-demand coverage, time to confirmed buyer, successful pickup rate and percentage of fulfilled orders.
- Failed collections, incidents, disputes, unexpected inventory discrepancies and platform operating cost.
- Measure avoided loss only with a credible baseline; never infer it solely from algorithm-generated recommendations.

1.4 Staged product releases

RELEASE	SCOPE	ACCEPTANCE
Hackathon MVP	Roles, harvests, demand, score, matches, offers, queue and demonstration transactions	One full authorized harvest-to-buyer journey
Field V1	Verified users, logistics, storage, incidents, notification outbox and reporting	Pilot transactions measured and audited
Regional V2	Analytics, optimization, multilingual access, financial integrations	Expansion only after evidence of value

02 | Brand and interface design

2.1 Color system

The visual language should communicate trust, agriculture and operating precision. Avoid generic farm-ecommerce clichés, 3D assets, neon, excessive gradients and heavy glassmorphism. Use crisp tables, generous whitespace, clear operational states and restrained accents.

FOREST GREEN
#103D30

HARVEST LIME
#BFE879

DEEP EMERALD
#19765A

WARM IVORY
#F7F8F2

PURE WHITE
#FFFFFF

SLATE CHARCOAL
#273630

SOFT SAGE
#E6EEE5

AMBER
#B45309

CRITICAL RED
#C2413B

TOKEN	HEX	ROLE
Forest Green	#103D30	Navigation, brand anchors, footers
Harvest Lime	#BFE879	Accent, select highlights
Deep Emerald	#19765A	Primary actions and confirmed states
Warm Ivory	#F7F8F2	Page background
Slate Charcoal	#273630	Primary content text
Soft Sage	#E6EEE5	Neutral surfaces / separators
Amber	#B45309	Attention and warning
Critical Red	#C2413B	Critical failures and blocked actions

2.2 Typography and design tokens

- Headlines: Manrope bold; interface: Inter. Bundle fonts via supported web font delivery; define system fallbacks.
- Content container 1360-1440px, sidebar ~248px, spacing on a 4/8px scale and card corners 12-16px.
- Mobile-first forms and tables, minimum 44px touch areas, keyboard focus rings and textual risk labels.
- Use date, quantity, location and status badges consistently; avoid color-only indicators.
- Make risk and matching explanations visible in the same screen as the recommended action.

2.3 Primary navigation

EXPERIENCE	PAGES / WORKSPACES
Public	Home; how it works; farmers; buyers; logistics; storage; impact; about; contact; auth
Farmer	Overview; farms; harvests; offers; orders; storage; shipment requests; reports; account
Buyer	Overview; requirements; marketplace; offers; orders; receipts; saved suppliers
Transport	Vehicles; availability; quote requests; jobs; delivery evidence; invoices
Storage	Facilities; calendars; reservation requests; inventory intake; incident reports
Coordinator	Overview; risk queue; buyer demand; matches; storage; logistics; orders; impact
Administration	Verification; users; organizations; configuration; audit; incident management

03 | Accounts, roles and access policy

ROLE	ALLOWED RESPONSIBILITIES
Farmer	Manage authorized farms/harvest lots; offers; own orders, receipts and loss reports
Buyer	Manage purchase requirements, offers and buyer-side delivery confirmation
Transport provider	Manage own vehicles, availability, assigned jobs and proof of delivery
Storage provider	Manage facilities, price/rules, calendar and reservations
Cooperative / agent	Manage only delegated farms, harvests and aggregation batches
Coordinator	Manage region-scoped cases, matching workflow and documented interventions
Admin	Review users, verification, disputes, settings and bounded oversight
Super Admin	Controlled global access with strong MFA and audit trails

Model identities through Supabase Auth plus profiles. Model organizations separately and join people through organization_memberships. A single person may participate in multiple organizations with different roles. Never infer organization membership from role labels alone.

3.1 Account lifecycle

- Registration → verified email/phone as applicable → profile → organization creation/invitation → role approval → onboarding.
- Privileged status (verified buyer/provider/cooperative) requires review and supporting evidence.
- Suspension, revocation, invitation expiration, onboarding updates and deactivation are auditable.
- Require MFA for administrative production accounts; verify user and organization permissions server-side on each write.

04 | Technology and infrastructure

LAYER	STACK	RATIONALE
Frontend	Next.js App Router, TypeScript	Unified application, server components and typed contracts
Styling	SCSS Modules + tokens	Maintainable branded components
Auth	Supabase Auth / @supabase/ssr	Cookie-based sessions and user identity
SQL access	postgres.js, Session Pooler	Parameterized queries and transactions
Storage	PostgreSQL + PostGIS	Relational integrity and geographic proximity
Media	Cloudinary optional	Private signed photograph uploads
Validation	Zod	Strict request validation
UI state	React; TanStack Query when useful	Server and interactive data state
Mapping	MapLibre and compliant tile provider	Display and route visualization
Testing	Vitest; Playwright	Logic, SQL integrations and E2E
Hosting	Vercel, Supabase	Simple Next.js deployment
Observability	Sentry and structured logs	Detect errors and transaction faults

4.1 Supabase Session Pooler

Obtain the actual connection string from Supabase Dashboard → Connect → Session pooler. Shared session-mode pooler typically uses port 5432; transaction-mode shared pooler typically uses port 6543. Do not assume a sample hostname is correct. Session-mode connections stay associated with a client; small connection pools must be sized and benchmarked for serverless concurrency.

```
DATABASE_URL="postgresql://postgres.PROJECT_REF:PASSWORD@POOLER_HOST:5432/postgres"
NEXT_PUBLIC_SUPABASE_URL="https://PROJECT.supabase.co"
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY="..."
CLOUDINARY_API_KEY="..."        # private, optional
CLOUDINARY_API_SECRET="..."     # private, optional
RESEND_API_KEY="..."            # optional
```

Important: A direct privileged SQL connection does not inherit the Supabase client user’s JWT context. Always authenticate the request and authorize the organization and resource in trusted server code. Enable and test RLS for tables exposed via Supabase browser/client APIs. Database passwords and privileged keys must never be prefixed NEXT_PUBLIC_.

4.2 Three-layer architecture



4.3 Module boundaries

Separate business logic from React pages. Use feature modules with validation schemas, repositories, services, server actions, API handlers, test fixtures and role checks. Keep SQL migrations version controlled. For large operations, asynchronous notifications should be processed from an outbox instead of inside the transaction that reserves produce.

05 | Functional module requirements

5.1 Public site and marketing

- Present a short agricultural problem narrative, operating process and verified benefits by stakeholder type.
- Separate farmers, buyers, transporters and storage-provider signup entry points.
- Explain how recommendations work and label all simulated or pilot statistics explicitly.
- Provide contact/privacy/terms pages and accessible account recovery.

5.2 Farmer workspace

- Manage profile, organization membership, one or more farms and approximate locations.
- Create draft harvest with crop, variety, grade, photos, dates, estimated quantity, unit and target price.
- Publish only after validation; support updates with versioned event history.
- Maintain multiple harvest lots and partial quantity allocation; clearly distinguish available, reserved, sold and lost stock.
- View ranked buyer demand, offer amounts, logistics estimates and explanations.
- Accept/reject/counter offers with deadlines; monitor collection, delivery and settlement records.
- Submit end-of-harvest outcome reports, inspections and disputes.

5.3 Buyer workspace

- Business details and verification status; multiple delivery destinations where authorized.
- Purchase requirements with crop, grade, quantity, maximum price, collection window, location and expiry.
- Browse compatible farm lots; submit and track offers; confirm purchase terms.
- Confirm received quantity and grade; flag discrepancies; attach evidence.
- Record payment references and review history, but do not self-confirm payment without trusted verification.

5.4 Transport provider

- Register vehicle capacity, service area, suitability (open/covered/insulated/refrigerated) and availability.
- Quote pickup/delivery jobs; accept assignments; record dispatched, collected, in-transit, delivered and failed statuses.
- Provide delivery evidence, actual charges, completion time and incident reasons.
- MVP may use coordinator-assisted bookings; full real-time vehicle tracking is explicitly out of scope.

5.5 Storage provider

- Register facility, usable capacity, compatible produce, conditions, prices and available date ranges.
- Issue and accept timed reservations; prevent capacity overbooking across overlapping periods.
- Track goods received/released and incidents, including deterioration, wrong quantity and late collection.
- Require a provider confirmation before displaying capacity as reserved.

5.6 Coordinator and administrator

- Risk queue with explainable priority, last evaluated time and recommended interventions.
- Review matches, generate quote requests, monitor delayed pickups and resolve unfulfilled orders.
- Verify organizations and providers; constrain staff actions to assigned scopes; track before/after changes.
- Configure catalogs, units, policies, score thresholds and escalation rules with versioned changes.
- Review impact metrics and distinguish forecasted outcomes from independently verified outcomes.

06 | Detailed operational workflows

6.1 Harvest-to-sale sequence

STEP	ACTION	SYSTEM GUARANTEE
01	Farmer creates draft	Authorized farm and validated quantity
02	Farmer publishes	Crop and dates eligible for discovery
03	Risk service evaluates	Score and reasons stored with version
04	Buyer demand matches	Hard constraints validated first
05	Buyer submits offer	Remaining quantity considered
06	Farmer accepts offer	Inventory atomically reserved
07	Logistics coordinated	Confirmation and pickup windows recorded
08	Receipt acknowledged	Discrepancies captured
09	Payment verified/recorded	Status based on trusted evidence
10	Impact measurement	Actual sold/lost quantities reconciled

6.2 Partial fulfillment and reversal

Example: 100 kg of tomatoes; Buyer A reserves 30 kg and Buyer B reserves 40 kg; 30 kg remains available. A competing request must not reserve more than the available balance. If Buyer B cancels before dispatch under the applicable rules, 40 kg may return to available stock; if dispatched, a controlled resolution process is required.

6.3 Exception handling

- Late pickups create operational incidents and trigger review.
- Produce deterioration and rejected quantities require inspection evidence or an explicit unverified declaration.
- Payment delays, order cancellation, duplicated retries and no-shows retain complete audit history.
- Reservations expire safely when time windows lapse; no background task may silently double-sell inventory.

07 | Decision-intelligence engine

7.1 Risk scoring v1: transparent rule-based model

Prototype formula: R = 0.30U + 0.25T + 0.20S + 0.15D + 0.10C. Every factor is normalized to a 0-100 severity value. U = uncommitted supply, T = urgency relative to crop handling time, S = storage inadequacy, D = expected logistics delay and C = reported condition risk. Proposed—not scientifically calibrated—weights.

FACTOR	INPUT / NORMALIZATION	WEIGHT
Uncommitted supply (U)	100 × uncommitted mass / eligible mass	30%
Time urgency (T)	Crop-specific window, harvest timing and elapsed time	25%
Storage inadequacy (S)	Availability and suitability of usable storage	20%
Delivery delay (D)	Expected pickup or delivery delay relative to window	15%
Produce condition (C)	Reported/inspected grade and condition deterioration	10%

Initial bands: 0–34 low; 35–64 moderate; 65–79 high; 80–100 critical. A pre-harvest score is a commercial-coordination urgency indicator, not a probability that produce will spoil. Keep crop handling assumptions versioned and explicitly inspectable.

7.2 Buyer matching

Step 1: Hard filters. Crop/grade compatible; demand unexpired; feasible delivery/pickup window; sufficient available inventory; permitted location; account standing; verified requirements. Exclude ineligible candidates before scoring.

MATCH WEIGHT	CRITERION
30%	Expected net farmer revenue
25%	Required delivery/pickup window compatibility
15%	Demand quantity compatibility
15%	Logistics feasibility

MATCH WEIGHT	CRITERION
15%	Documented buyer reliability (unknown if insufficient history)

Step 2: Rank with explanations. Return buyer identity/status, requested crop and quantity, location, offer, logistics estimate, expected farmer proceeds, deadline, fit reasons and missing assumptions. Never invent a buyer reliability score for a new account.

7.3 Fulfillment option evaluation

Net return = expected sales revenue − transport expense − storage expense − other documented transaction costs. Compare immediate local sale, higher-price distant sale, validated temporary storage, joint transport and alternative processing pathways. State unavailable pricing inputs, time constraints and risks; prioritize feasible alternatives, not merely the highest gross selling price.

7.4 AI usage boundaries

Generative AI may later explain scores, translate instructions, summarize market documents or triage anomalies. It must not autonomously commit stock, confirm payments, book services or infer scientific spoilage certainty. The hackathon prototype is fully viable with deterministic algorithms and validated business logic.

08 | Data model and schema blueprint

8.1 Core entities

DOMAIN	TABLES
Identity	profiles, organizations, organization_memberships, verification_requests
Agriculture	farms, crops, crop_varieties, units, harvests, harvest_lots
Inventory	inventory_movements, inventory_reservations
Demand & matching	buyer_requirements, match_recommendations, purchase_offers
Commerce	orders, order_items, payment_records, incidents
Fulfillment	transport_vehicles, shipments, storage_facilities, storage_reservations
Intelligence	risk_assessments, algorithm_configurations, recommendation_actions
Platform	notifications, notification_outbox, audit_logs, impact_measurements

8.2 Data design standards

- Use UUID primary keys, non-null references where required, check constraints, foreign keys and uniqueness constraints.
- Store money as integer kobo (or other currency minor units) and include an ISO currency code.
- Keep reported local quantity and unit plus measured kilograms when verified; baskets are not universal units.
- Use PostGIS geography(Point, 4326) for coordinates, with access-scoped exact locations.
- Use timestamptz for events and explicit timezone treatment for local dates.

- Retain event/audit history and separate reservations from physical stock movements.
- Index crop/date/status, actor/resource, organization membership and geospatial lookup paths.

8.3 Initial migration (representative, executable SQL)

```
create extension if not exists pgcrypto;
create extension if not exists postgis with schema extensions;

create type public.organization_type as enum (
  'farmer', 'buyer', 'cooperative',
  'transporter', 'storage_provider'
);

create type public.harvest_status as enum (
  'draft', 'published', 'partially_allocated',
  'allocated', 'completed', 'cancelled'
);

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  phone text,
  locale text not null default 'en-NG',
  created_at timestamptz not null default now()
);

create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  type public.organization_type not null,
  verification_status text not null default 'pending'
    check (verification_status in
      ('pending','verified','rejected','suspended')),
  created_at timestamptz not null default now()
);

create table public.organization_memberships (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid not null references public.organizations(id),
  user_id uuid not null references public.profiles(id),
  role text not null,
  status text not null default 'active'
    check (status in ('invited','active','suspended')),
  unique (organization_id, user_id)
);

create table public.crops (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  category text,
  created_at timestamptz not null default now()
);
```

```
create table public.units (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,
  label text not null,
  measurement_type text not null
    check (measurement_type in ('mass','count','volume'))
);

create table public.farms (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid not null references public.organizations(id),
  name text not null,
  location extensions.geography(Point,4326),
  address_description text,
  created_at timestamptz not null default now()
);

create table public.harvests (
  id uuid primary key default gen_random_uuid(),
  farm_id uuid not null references public.farms(id),
  crop_id uuid not null references public.crops(id),
  estimated_harvest_date date not null,
  actual_harvest_date date,
  total_quantity numeric(14,3) not null check(total_quantity > 0),
  unit_id uuid not null references public.units(id),
  status public.harvest_status not null default 'draft',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index farms_location_gix on public.farms using gist(location);
create index harvests_crop_date_idx
  on public.harvests(crop_id, estimated_harvest_date);
create index harvests_status_idx on public.harvests(status);

alter table public.profiles enable row level security;
alter table public.organizations enable row level security;
alter table public.organization_memberships enable row level security;
alter table public.crops enable row level security;
alter table public.units enable row level security;
alter table public.farms enable row level security;
alter table public.harvests enable row level security;
```

Implementation warning: Enabling RLS without suitable policies denies ordinary data access; intentional during migration preparation. Apply tested, least-privilege policies before client access. This starter migration omits transactional entity migrations and is not the entire production database.

09 | APIs and backend contracts

VERB	ENDPOINT	PURPOSE
GET / POST	/api/harvests	List/create authorized harvests
GET / PATCH	/api/harvests/:id	Read/update authorized harvest
POST	/api/harvests/:id/publish	Validate and publish
GET	/api/harvests/:id/risk	Read current score and factors
GET	/api/harvests/:id/matches	View ranked eligible demand
POST	/api/buyer-requirements	Create buyer demand
POST	/api/offers	Place offer with expiration
POST	/api/offers/:id/accept	Atomically accept and reserve
POST	/api/offers/:id/reject	Reject offer
GET	/api/orders	Scope orders to membership
POST	/api/orders/:id/confirm-receipt	Log delivery acceptance
POST	/api/transport/quotes	Request/submit transport quote
POST	/api/storage/reservations	Request storage window
GET	/api/operations/overview	Coordinator operations queue
GET	/api/impact/summary	Auditable impact summary
POST	/api/media/signature	Signed restricted image upload

9.1 Backend contract and concurrency

- Require verified Supabase identity, organization-scoped authorization, Zod validation, and per-route rate limits.
- Standardize structured success/error objects, typed problem codes, request IDs, pagination, and cursor handling.
- Use SQL transactions and row locking or another proven atomic strategy for reservation acceptance.
- Use idempotency keys for offers/orders and transaction-level uniqueness guards for retries.
- Never consider an unverified payment reference equivalent to settled funds.

```
{
  "success": true,
  "data": {
    "harvestId": "sample-harvest-id",
    "status": "published",
    "risk": {
      "score": 72,
      "level": "high",
      "modelVersion": "rules-v1",
      "factors": [
        { "name": "uncommitted_supply", "score": 85 },
        { "name": "storage_inadequacy", "score": 70 }
      ]
    },
    "matchedBuyers": 3,
    "lastAssessedAt": "2026-10-18T12:00:00Z"
  }
}
```

10 | Repository structure and initial configuration

```
harvestos/
  public/images/                public/icons/
  src/app/
    (marketing)/                 (auth)/auth/
    (platform)/farmer/           (platform)/buyer/
    (platform)/transport/        (platform)/storage/
    (platform)/operations/       (platform)/admin/
    api/harvests/                api/buyer-requirements/
    api/offers/                  api/orders/
    api/transport/               api/storage/
    api/impact/                  api/media/
  src/components/ui/            src/components/forms/
  src/components/layout/        src/components/charts/
  src/features/auth/            src/features/organizations/
  src/features/farms/           src/features/harvests/
  src/features/demand/          src/features/intelligence/
  src/features/matching/        src/features/inventory/
  src/features/orders/          src/features/logistics/
  src/features/storage/         src/features/notifications/
  src/features/impact/          src/lib/supabase/
  src/lib/db/                   src/lib/auth/
  src/lib/validations/          src/lib/cloudinary/
  src/styles/_tokens.scss      src/styles/_mixins.scss
  src/styles/globals.scss       src/types/
  supabase/migrations/          supabase/seed.sql
  tests/unit/                   tests/integration/
  tests/e2e/                    docs/
```

10.1 Package installation

```
npx create-next-app@latest harvestos --typescript --eslint --app --src-dir
cd harvestos
npm install sass @supabase/supabase-js @supabase/ssr postgres zod \
  lucide-react @tanstack/react-query recharts maplibre-gl
npm install -D vitest @playwright/test @types/node
# Optional: npm install cloudinary next-cloudinary
```

10.2 Session-mode PostgreSQL connection module

```
import "server-only";
import postgres from "postgres";

if (!process.env.DATABASE_URL) {
  throw new Error("DATABASE_URL is missing");
}

export const sql = postgres(process.env.DATABASE_URL, {
  max: 2,
  idle_timeout: 20,
  connect_timeout: 10,
});
```

Connection pool max=2 is a conservative starting point per runtime instance, not a production universal constant. Benchmark concurrent deployments and pooler usage. In heavily serverless conditions, compare session versus transaction pooler appropriately.

10.3 SCSS foundation

```
$forest: #103d30;
$lime: #bfe879;
$emerald: #19765a;
$ivory: #f7f8f2;
$white: #ffffff;
$charcoal: #273630;
$sage: #e6eee5;
$amber: #b45309;
$critical: #c2413b;

:root {
  --color-primary: #{$forest};
  --color-accent: #{$lime};
  --color-action: #{$emerald};
  --color-background: #{$ivory};
  --color-surface: #{$white};
  --color-text: #{$charcoal};
  --color-border: #{$sage};
  --color-warning: #{$amber};
  --color-danger: #{$critical};
  --radius-card: 14px;
  --radius-control: 10px;
  --container-width: 1400px;
}
* { box-sizing: border-box; }
body { background: var(--color-background); color: var(--color-text); margin: 0; }
```

11 | Security and privacy standards

11.1 Security controls

- Use Supabase Auth and secure server-session handling; enforce RBAC plus object/organization authorization.
- Never trust hidden navigation, role names, or client-side checks as enforcement.
- Enable RLS and test all direct browser/client Supabase exposure.
- Require strong MFA for privileged production admin accounts.
- Limit access to exact farm locations, phones, documents, prices and settlement evidence.
- Apply throttling, signed uploads, server-side file validation, and private media access where appropriate.

- Maintain immutable-style audit history for sensitive changes and incident handling.
- Use idempotency and atomic transactions to prevent double selling and duplicate charges.

11.2 Privacy and data stewardship

Prepare privacy notices, lawful processing documentation, access-request workflow, retention rules, business-partner obligations and incident response aligned with Nigeria's applicable data protection requirements (including the Nigeria Data Protection Act 2023). Review external service-provider and cross-border data flows before production. A farmer's exact residence or farm coordinates should never automatically become a publicly searchable attribute.

11.3 Reliability

- Separate development, staging and production projects; use formal migrations and reviewed release steps.
- Use error tracking, structured logs and database connection visibility.
- Implement an outbox for emails/SMS so notification failures cannot replay orders.
- Maintain reliable backups, tested restore procedures, service retry bounds and rollback playbooks.
- Record failed pickups and disputes with reasons, timestamps and owner assignments.

12 | QA, acceptance and accessibility

TEST CATEGORY	SCENARIO	REQUIRED RESULT
Access	Buyer edits another farmer's lot	Denied
Inventory	Two offers compete for final quantity	Cannot oversell
Matching	Crop or grade conflicts	Excluded
Risk	Inputs change	Versioned score refreshed
Offer	Double submission / retry	One accepted order
Storage	Overlapping capacity reservations	No overbooking
Logistics	Failed pickup	Incident opens with evidence
Payment	Unverified claim	Not recorded as confirmed
Impact	Only match exists	No rescue counted
Media	Malicious or oversized upload	Rejected
Mobile	320px viewport registration	Task remains usable

Testing: Vitest for pure scoring and matching calculations, database integration tests for atomic reservations and permissions, Playwright for full user journeys, keyboard and contrast checks, manual low-bandwidth/device tests, and field acceptance tests with representatives of each operational role.

12.1 Formal MVP acceptance criteria

- Farmer creates a farm, publishes a valid harvest and sees accurate remaining stock.

- Buyer creates compatible demand, submits an offer and receives explicit confirmation status.
- Farmer accepts with correct atomic allocation; conflicting offers cannot oversell the harvest.
- Operations queue shows explainable urgency and eligible matching options.
- Coordinator can record fulfillment and discrepancies without overwriting history.
- Dashboards do not present simulated, pledged or algorithmic quantities as actual food saved.
- All privileged functions are protected by tests demonstrating denial to unauthorized roles.

13 | Delivery plan and release cadence

PHASE	WINDOW	DELIVERABLE
Discovery & UX	Week 1	Interviews, PRD, role journeys, wireframes, ERD, backlog
Foundation	Week 2	Next.js, database, migrations, authentication, RBAC, components
Core commerce	Weeks 3–4	Farms, harvests, demand, offers, inventory and orders
Intelligence	Week 5	Risk, matching, coordinator interventions, explanations
Fulfillment	Weeks 6–7	Transport jobs, storage, reservations, incidents
Administration	Week 8	Impact reporting, staff configuration, notifications, auditing
Hardening	Weeks 9–10	Security tests, performance, training, pilot release candidate

13.1 HackJos 48-hour build sequence

WINDOW	WORK
0–8 hours	Define a verified problem, scope a tomato workflow, schema and front-end shell
8–20 hours	Harvest forms, buyer requirements, matching and transaction path
20–32 hours	Explainable urgency engine, price/logistics comparison, operations dashboard
32–40 hours	Test low demand, delays, constraints, inventory concurrency and UI
40–48 hours	Demo stabilization, presentation, disclosures of simulated data, final pitch

13.2 Pre-event must-build checklist

- Repository and coding conventions; Supabase Session Pooler connection; initial migrations.
- Auth, onboarding and scoped workspaces for farmer/buyer/coordinator.
- Crop/harvest forms, buyer requirements, matching, inventory reservations.
- Urgency score with explicit explanation and configurable assumptions.
- Coordinator queue, order outcome dashboard and seeded clearly fictional scenario.
- One fully tested end-to-end journey and polished three-minute demo.

13.3 Explicitly postponed features

Defer live payments, loans, crop insurance, independent native mobile apps, IoT monitoring, advanced fleet tracking, autonomous transaction execution, sophisticated spoilage ML and globally optimized delivery routing until the core transactional system is stable.

14 | Release engineering and deployment

- Use separate environment credentials for development, staging and production.
- Pull-request review → automated tests → migration review → staging deployment → E2E tests → production migration → production smoke test.
- Prepare an explicit rollback and backup/restore procedure before schema-breaking changes.
- Audit logs and access policies should be reviewed as part of deployment acceptance.
- Monitor API latency, failures, database pool utilization, reconciliation discrepancies and notification queue age.

15 | Field research and economic model

15.1 Interview plan

Proposed discovery sample: at least five farmers, three produce traders, two bulk buyers and two transport/storage operators. Ask about specific recent harvests and transactions, rather than general hypothetical difficulties.

- What crop and quantity did you harvest most recently; what portion sold, spoiled or remained unsold?
- When was the buyer confirmed and how was the selling price established?
- What were the actual transport, storage, handling and transaction costs?
- Why did any promised pickup or payment fail?
- What information would help providers plan supply, vehicle capacity and buyer demand?
- Would an authorized agent be more trustworthy or accessible than a purely self-service app?

15.2 Proposed pilot

Recruitment hypothesis: one crop, one Plateau farming cluster, about 10–20 farmers, a handful of buyers and a small number of verified transport/storage providers. The figures are planning targets, not committed participants.

MEASURE	PROPOSED VALIDATION TARGET
Completed transactions	At least 10 verified pilot transactions
Onboarding usability	At least 80% can complete with reasonable assistance
Stock integrity	100% of tested inventory scenarios reconcile
Critical integrity incidents	Zero unexplained over-allocation
Loss reduction	Evaluate against measured baseline, not invented percentage
User feedback	Track user failure cases and repeated workflow friction

15.3 Potential revenue strategy

- Small fee charged only on verified completed transactions when user economics support it.
- Buyer subscription for recurring procurement and demand planning.
- Cooperative plan for aggregation and multi-farm management.
- Documented logistics coordination service fee.
- Enterprise analytics built from lawfully managed, appropriately anonymized aggregate data.

Avoid charging vulnerable farming participants merely to publish a listing before meaningful value is demonstrated.

16 | International product strategy

The international opportunity is a repeatable decision infrastructure for fragmented agricultural systems, rather than a claim that food marketplaces or cold-storage booking are new. Test measurable improvements in transaction completion, actual loss and verified income before making cross-border impact claims.

- Support localized crops, grades, handling windows, quality rules and farming practices.
- Model currency, date/time and unit conversions explicitly; preserve original local units.
- Apply country-specific privacy and financial rules before deployment or data sharing.
- Expose documented, authorized partner APIs only after tenant-isolation tests.
- Publish clear study methods so research partners can evaluate real-world outcomes.

17 | Risk register and mitigations

RISK	WHY IT MATTERS	MITIGATION
Weak actual buyer demand	Matching cannot generate buyers	Secure buyers before growth promises
Incorrect basket weights	False capacity and value calculations	Require unit-specific conversions
High spoilage uncertainty	Bad rescue recommendations	Label assumptions; add expert inspection
Overselling	Breaks trust and settlement	Transactions, locks and idempotency
Poor connectivity	Rural users unable to participate	Agent mode and low-bandwidth UI
Transport no-shows	Fulfillment fails after match	Provider confirmations and incident tracking
Data privacy breach	Loss of trust and legal exposure	Least privilege and minimized location access
Too broad MVP	Demo remains incomplete	One crop, one end-to-end workflow

18 | Research, technical references and assumptions

Primary source links and documentation relevant to implementation:

HackJos 2026 challenge: https://www.hackjos.com.ng/challenge

Next.js Authentication Guide: https://nextjs.org/docs/app/guides/authentication

Supabase database connection options: https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase SSR Auth: https://supabase.com/docs/guides/auth/server-side

Cloudinary Next.js Upload Guide: https://cloudinary.com/documentation/nextjs_image_and_video_upload

FAO Food Loss Indicator: https://www.fao.org/sustainable-development-goals-data-portal/data/indicators/1231-global-food-losses/en

ColdHubs: https://coldhubs.com/cold-connect-app

Data honesty: Sample dashboards, business metrics, matching weights, scoring thresholds, timelines and pilot targets in this document are proposals for prototype development. They are not live operational data or a guarantee of agricultural impact.

19 | Project definition of done

A real farmer can join an approved organization, publish a legitimate harvest, receive an explainable urgency assessment and buyer matches, accept an authorized offer with atomic stock reservation, arrange fulfillment, record verified receipt and reconcile the resulting sale, unsold quantity and losses—without manual database edits. A coordinator can inspect each state transition. Unauthorized actors cannot alter it. Only verified outcomes appear as achieved impact.

Recommended engineering order: (1) record system, (2) consistent commerce workflow, (3) transparent intelligence, (4) provider coordination, (5) measurement, (6) advanced AI. The system should be credible before it is complex.

END OF MASTER ROADMAP